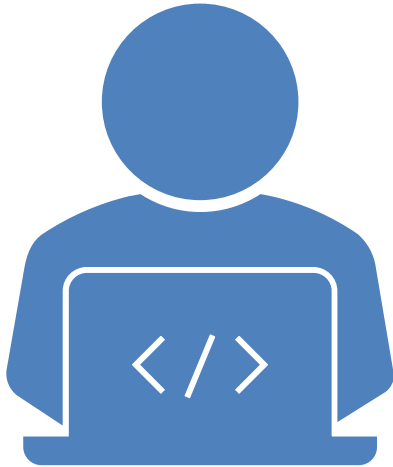


Comparative Sentiment Analysis on Twitter Using Classical, Deep Learning and Transformer-Based Models



Submitted By: Arpan Neupane
Email: Neupane-a1@ulster.ac.uk

Introduction

- Project explores sentiment analysis on Twitter data about people's opinions by classifying them as positive, neutral and negative.
- To tackle, Comparison between classical ML methods (Naïve Bayes & Random Forest), deep learning , and transformer-based models
- The key parts of this work was looking at transformers in two different ways: frozen, where just use pre-trained embeddings, and fine-tuned, where I adapted the whole model to the Twitter dataset.
- Key objective: identify most effective model for sentiment classification.

Research Aims & Objectives

Compare	Compare performance of multiple ML approaches.
Investigate	Investigate impact of frozen vs. fine-tuned transformer embeddings.
Identify	Identify trade-offs between accuracy, computational efficiency and interpretability.
Provide	Provide reproducible benchmarks for Twitter Sentiment classification.

Dataset (Kaggle)



Sources: Sentiment Analysis Dataset ([ABDELMALEK ELADJELET, Kaggle](#))



Size: *~ 48,000 tweets/ comments.*



Sentiment Classes: Positive, Neutral and Negative



Challenges: Class imbalances (Negative < Neutral < Positive)

Dataset & Preprocessing

- Removal of links, hashtags, usernames, etc. using Stop-word removal.
- Tokenisation:
 - TF-IDF (classical models) to represent tweets as weighted word features.
 - Word embeddings (LSTM/BiLSTM) to capture more semantic meaning of the words.
 - Subword tokenisation (BERT, RoBERTa) for breaking text into smaller pices to handle rare or unseen words more effectively.
- Sequence padding (100 for LSTM, 128 for transformers) to ensure all tweets had the same length.

Models Implemented

Classical ML (Served as interpretable baselines):

- Naïve Bayes: Lightweight, fast
- Random Forest: Robust ensemble

Deep Learning (implemented in Keras/ TensorFlow):

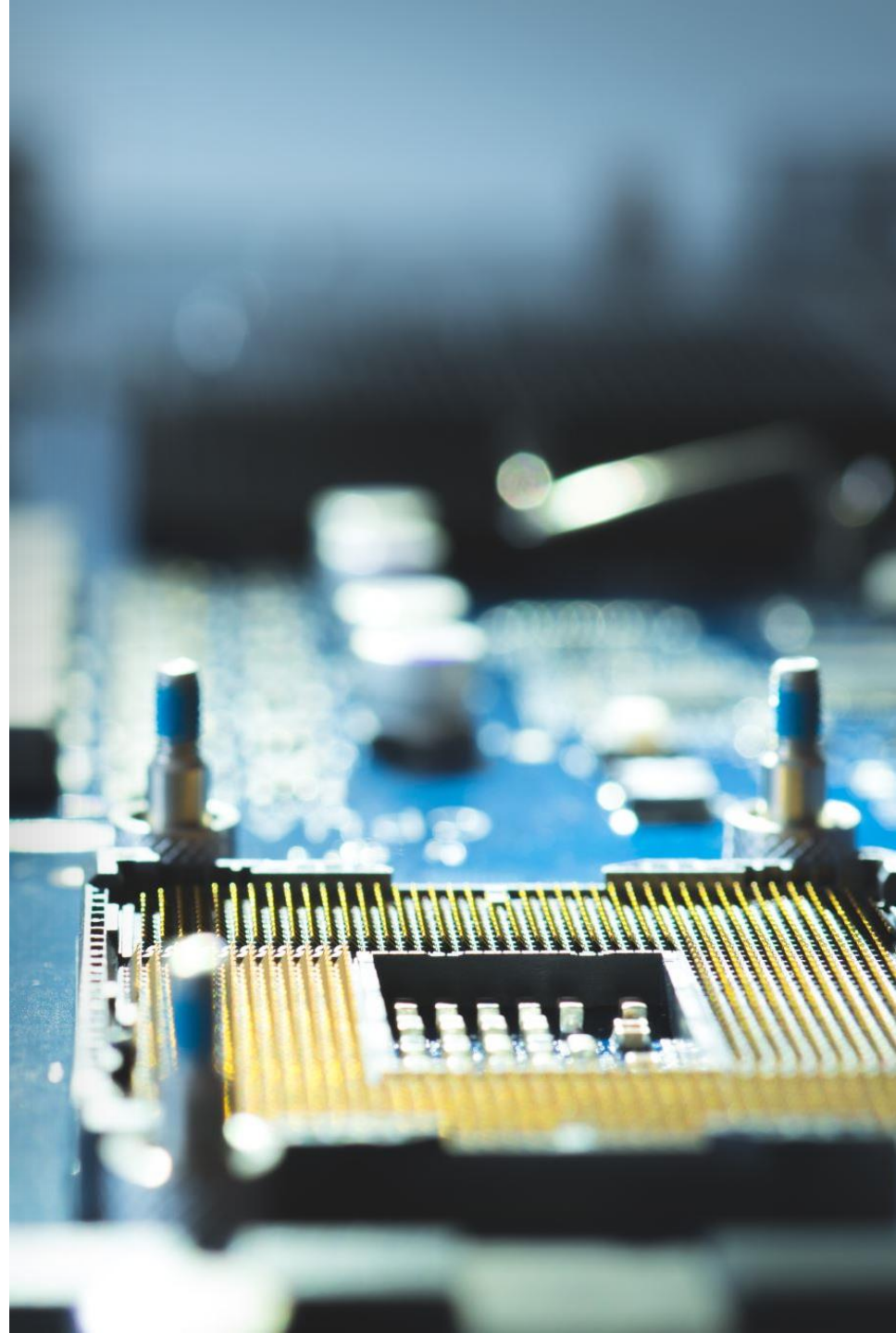
- LSTM: Struggled with imbalance
- BiLSTM: Captured bidirectional context

Transformers (implemented with HuggingFace Transformers i.e. PyTorch)

- BERT (Frozen & Fine-Tuned)
- RoBERTa (Frozen & Fine-Tuned)

Training Environment

- Hardware: Apple M1 chip, 8- core CPU/GPU
- Software: Python 3.9.6, TensorFlow 2.19, Keras 3.10, PyTorch 2.7.1, HuggingFace Transformers



Results Overview

Classical Models:

- Naïve Bayes: 68% accuracy
- Random Forest: 84% accuracy

Deep Learning:

- LSTM: 43% accuracy (biased)
- BiLSTM: 84% accuracy

Transformers:

- Frozen BERT: 48.6%
- Fine-Tuned BERT: 85.8%
- Frozen RoBERTa: 58.1%
- Fine-Tuned RoBERTa: 87.8% (Best)

Results Overview

Model	Accuracy	Notes
Naïve Bayes	68%	Struggled with neutral/negative balance.
Random Forest	84%	Competitive, handled neutral/positive well.
LSTM	43%	Collapsed, biased toward positives.
BiLSTM	84%	Strong sequential learner, balanced results.
Frozen BERT	48.6%	Collapsed, predicted positives disproportionately.
Fine-tuned BERT	85.8%	Strong performance, balanced across classes.
Frozen RoBERTa	58.1%	Better than frozen BERT, but weak overall.
Fine-tuned RoBERTa	87.8%	Best-performing model across all metrics.

Model Performance Comparison

```

Experiment: Full Fine-Tuning

Using device: mps
Some weights of RobertaForSequenceClassification were not initialized from the model checkpoint at roberta-base and are newly initialized: ['classifier.dense.bias', 'classifier.dense.weight', 'classifier.out_proj.bias', 'classifier.out_proj.weight']
You should probably TRAIN this model on a down-stream task to be able to use it for predictions and inference.
Epoch 1: 100%
Epoch 2: 100%
Epoch 3: 100%
12833/12833 [3:42:02<00:00, 1.11s/it, loss=0.9232]
12833/12833 [3:56:02<00:00, 1.15s/it, loss=0.3761]
12833/12833 [5:12:22<00:00, 1.56s/it, loss=0.2672]

Accuracy: 0.8775

Classification Report:
      precision    recall  f1-score   support

     0       0.84       0.83       0.84      11019
     1       0.87       0.87       0.87      16512
     2       0.90       0.91       0.90      20602

 accuracy      0.87         0.88      48133
  macro avg       0.87       0.87       0.87      48133
 weighted avg       0.88       0.88       0.88      48133

Training Time: 45867.38 seconds

===== FINAL COMPARISON =====
Frozen RoBERTa ~ Accuracy: 0.5807, Training Time: 14230.54s
Full Model ~ Accuracy: 0.8775, Training Time: 45867.38s
Accuracy Improvement from Freezing to Full: 0.2968
Misclassification pattern differences can be further analyzed in the confusion matrices above.

Results Summary:
      accuracy      time
Frozen RoBERTa    0.580683  14230.538669
Full Fine-Tuning  0.877227  45867.379711
arpan@Arpans-MacBook-Pro ~ %

```

```

=== Tuning Naive Bayes ===
Best parameters: {'alpha': 0.1}
Best macro-F1: 0.6543

```

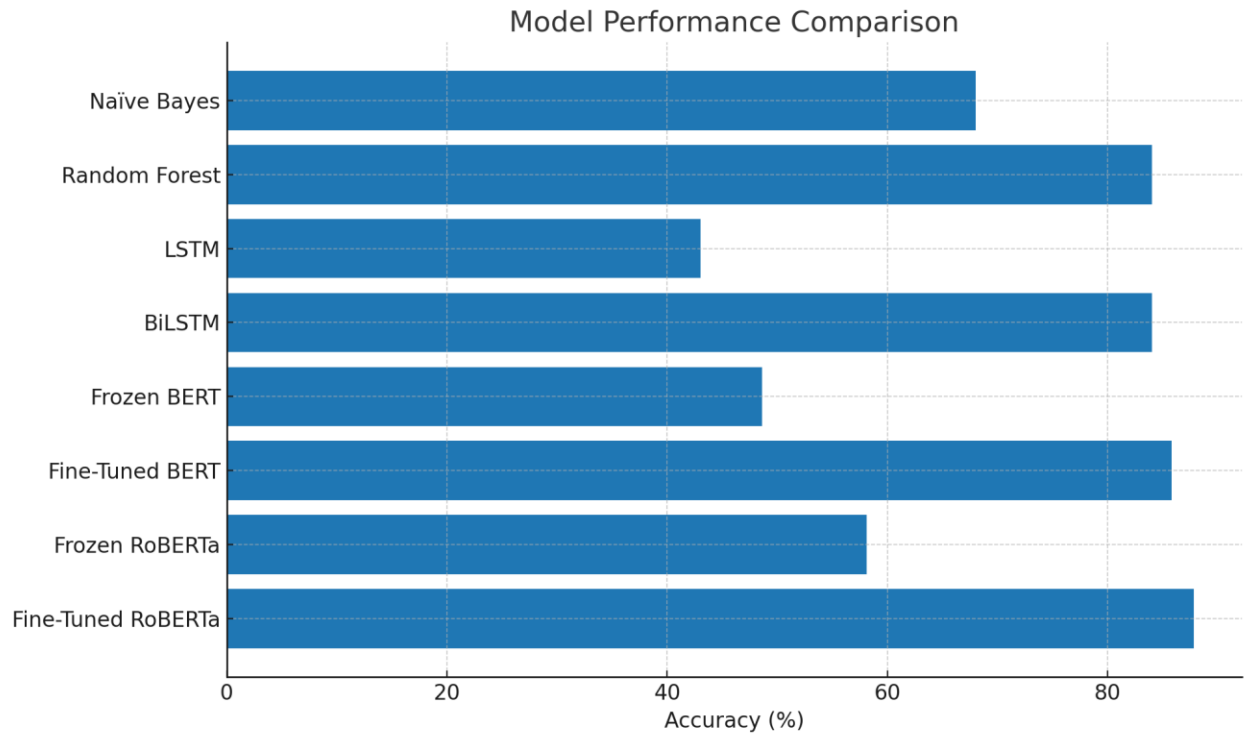
```

Classification Report:
      precision    recall  f1-score   support

     0       0.72       0.52       0.61      11021
     1       0.67       0.63       0.65      16556
     2       0.67       0.81       0.73      20609

 accuracy      0.68         0.68      48186
  macro avg       0.69       0.65       0.66      48186
 weighted avg       0.68       0.68       0.68      48186

```



Discussion

- • Fine-tuning is essential for transformer models
- • RoBERTa > BERT due to larger training corpus & optimized pre-training
- • Classical models competitive but lack contextual understanding
- • Negative sentiment detection remains a challenge due to class imbalance



Conclusion & Future Work



- Best Model: Fine-Tuned RoBERTa (87.8% accuracy)



- Transformers significantly outperform other approaches



- Future Work:



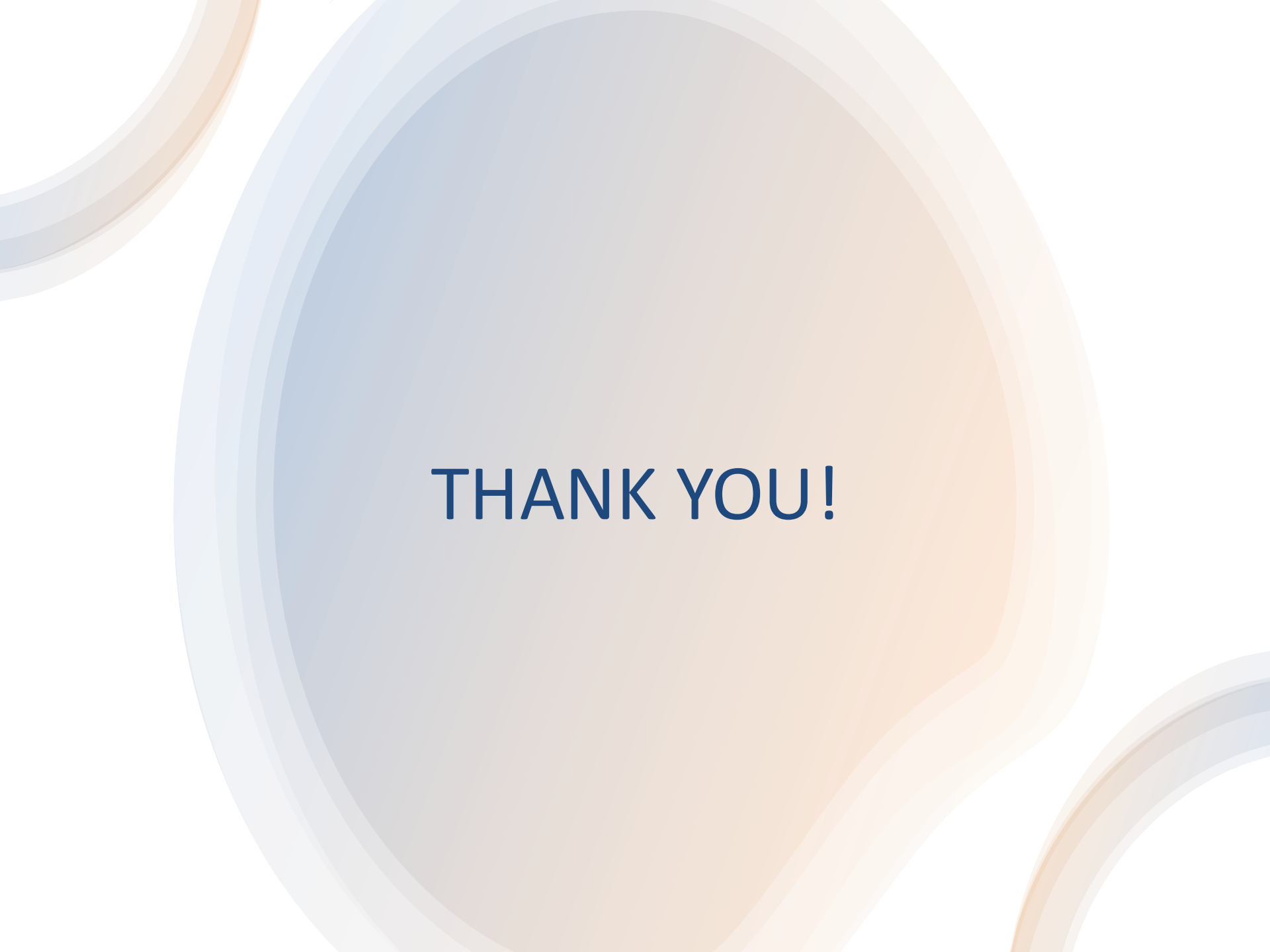
- Address class imbalance via augmentation/weighting



- Explore larger transformer architectures (e.g., XLNet, GPT)



- Test cross-domain generalisation



THANK YOU!